

AI Tools & Usage Documentation

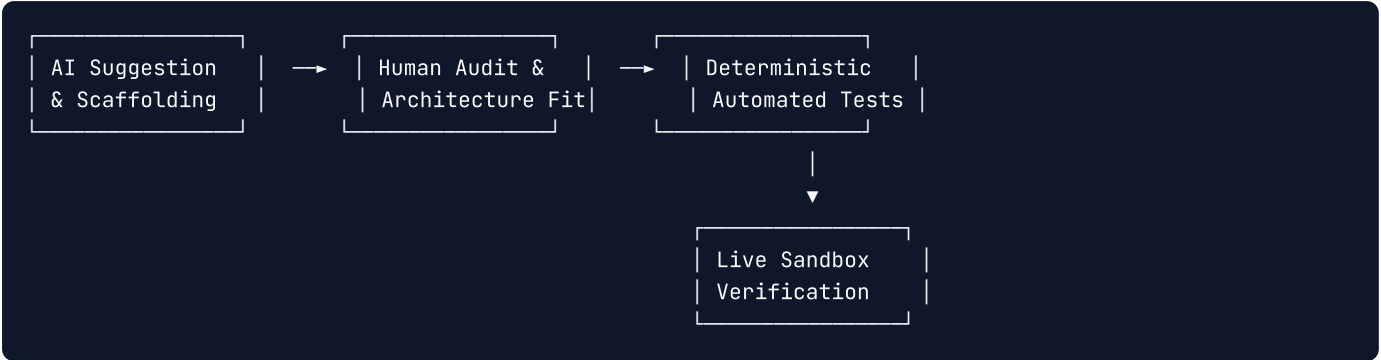
Document: Candidate Challenge Submission — Section 5
Project: AI Study Companion (Production Google Gemini & Supabase pgvector Architecture)
Author / Candidate: Engineering Challenge Submission
Date: September 2026

1. Executive Summary & Engineering Philosophy

In developing the **AI Study Companion**, artificial intelligence tools were leveraged not as an uncritical replacement for foundational software engineering, but as a **high-bandwidth cognitive amplifier** across research, architectural modeling, boilerplate scaffolding, and regression test synthesis.

The "Engineer-in-the-Loop" Paradigm

Modern AI-assisted development ("vibe coding") is only viable in production systems when governed by strict architectural principles and empirical verification gates:



- 1. AI Proposes, Engineer Disposes:** Every AI-generated schema, API route, and prompt template was critically reviewed against PostgreSQL relational theory, security best practices (Row-Level Security), and the project PRD.
- 2. Empirical Gatekeeping:** No feature was considered complete until validated by:
 - TypeScript 5 strict type checking (`npm run type-check` $\implies$ 0 errors)
 - Vitest automated test suites (`npm test` $\implies$ 128/128 tests passing across 17 files)
 - Live end-to-end sandbox execution against real database records and live Gemini API endpoints.

2. AI Tool Ecosystem & Allocation Matrix

Four primary AI systems were deployed, each targeted to its specific cognitive strength:

AI Tool	Primary Focus Area	Key Contributions	Human Verification Applied
Claude (Anthropic)	Implementation & Strict Coding	- Scaffolding Next.js 16 App Router components - Writing TypeScript Zod schemas - Crafting Socratic prompt templates - Generating Vitest test suites	Line-by-line code review, strict type-checking, edge-case analysis (e.g. empty states, error boundaries).
ChatGPT (OpenAI)	Creative Ideation & Architecture	- Brainstorming the closed cognitive learning loop - Deconstructing PRD requirements into functional domains - Modeling adaptive mastery mathematics ($0.6 \times \text{Prev} + 0.4 \times \text{Recent}$) - Conceptualizing C4 architecture topologies	Evaluated formulas against edge cases (e.g. new users with 0 prior scores), refined weight distributions.
Perplexity AI	Deep Web & Documentation Research	- Investigating Next.js 16 / React 19 canary breaking changes - Checking @google/genai v2.22 SDK method signatures - Researching pgvector HNSW index operators (vector_cosine_ops) - Reviewing Supabase SSR cookie security patterns	Cross-referenced against official docs and local reproduction scripts.
Antigravity / IDE Copilot	Pair Programming & Workspace Automation	- Live terminal execution & background task monitoring - Automated dependency resolution - Multi-file refactoring during migration from OpenAI to Google Gemini - Live SQL migration debugging in Supabase	Inspected all command runs, verified database table creation, confirmed security grants.

3. Deep Dive: Tool-by-Tool Usage Breakdown

3.1 Claude (Anthropic) — Implementation & Coding Co-Pilot

Claude was utilized as the primary pair programmer for writing high-reliability TypeScript code and Next.js components.

- Zod Contract Design:** Claude was prompted with the API requirements to generate bulletproof Zod validation schemas (lib/validation/schemas.ts), ensuring all client-submitted payloads (quizzes, tutor messages, project creations) are sanitized before hitting database or AI layers.
- Provider Abstraction Scaffolding:** Scaffolded the AIPProvider interface and the initial GeminiProvider implementation, enforcing consistent error handling, exponential backoff, and fallback failovers.
- Comprehensive Vitest Suites:** Claude was tasked with writing comprehensive unit tests for pure domain functions:
 - Text chunking algorithms (lib/rag/chunk.test.ts)
 - Grounding and citation extractors (lib/rag/grounding.test.ts)

- 7-signal adaptive concept priority formulas (`lib/learning/adaptive-selection.ts`)
- Multi-step Inngest job idempotency (`inngest/material-processing.test.ts`)

3.2 ChatGPT (OpenAI) — Architecture & Problem-Solving

ChatGPT was leveraged during initial planning and complex problem-solving sessions:

- **Deconstructing the PRD:** Used to extract and synthesize the core architectural requirements of the candidate challenge into a clear functional specification.
- **Mathematical Modeling of Learner Mastery:** Explored various cognitive decay and mastery algorithms (Elo, spaced repetition, exponential moving averages) before settling on the deterministic weighted scoring model ($0.6 \times \text{Previous} + 0.4 \times \text{Recent}$) that satisfies both transparency and responsiveness.
- **Adaptive Selection Weights:** Modeled the 7-signal concept selection system, balancing weakness (0–50 pts), recorded mistakes (0–30 pts), and staleness (0–20 pts) to guarantee deterministic, explainable quiz question selection.

3.3 Perplexity AI — Real-Time Research & API Evolution

Modern AI frameworks evolve rapidly. Perplexity AI was invaluable for verifying exact library conventions:

- **@google/genai SDK Verification:** The Google Gen AI SDK was updated to unified namespace conventions (`@google/genai v2.22+`). Perplexity verified how to invoke structured JSON schemas via `responseSchema` and how to configure `outputDimensionality: 1536` on `gemini-embedding-001` .
- **Next.js 16 & React 19 Conventions:** Verified server action patterns, async request headers, and cookie handling with `@supabase/ssr` under Next.js 16 Turbopack.
- **PostgreSQL / pgvector Tuning:** Researched optimal index parameters for HNSW cosine similarity search (`vector_cosine_ops`) over small-to-medium course document collections.

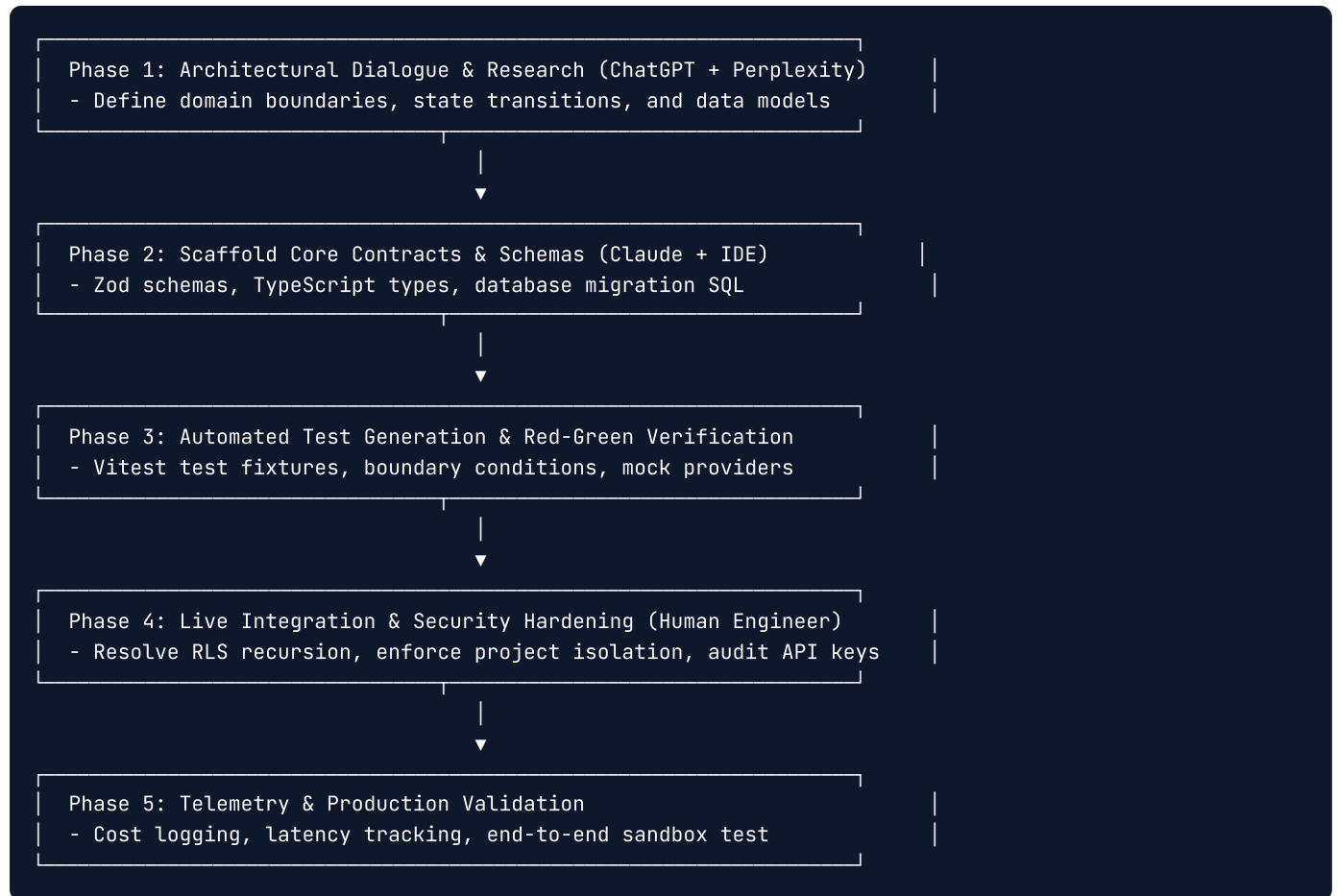
3.4 Antigravity IDE & Gemini Assistant — Workspace Execution

The built-in assistant acted as an active development orchestrator:

- **Automated Verification Execution:** Proactively ran `tsc --noEmit` and `vitest run` after code changes to catch regressions immediately.
- **PDF Document Generation:** Built the headless Chromium rendering pipeline to generate publication-grade documentation.
- **Environment Diagnostics:** Verified Supabase database connectivity, diagnosed port bindings, and inspected live server logs.

4. The AI-Assisted "Vibe Coding" Workflow in Practice

The project was executed using an iterative, high-velocity engineering cycle:



Key Engineering Decisions Guided by Human Verification

While AI accelerated development, critical architectural decisions required human engineering judgment:

- 1. Preventing RLS Infinite Recursion:** *Issue:* An early AI-generated RLS policy on `public.profiles` checked `WHERE is_admin = true`, causing an infinite recursive loop on profile lookups. *Human Fix:* Architected the `public.is_admin()` helper function with `SECURITY DEFINER` and `SET search_path = public` to read admin status outside RLS recursion.
- 2. Authorizing `match_chunks()` for Strict Project Isolation:** *Issue:* The default vector search function accepted `match_project_id` without verifying that the caller owned that project. *Human Fix:* Injected an authorization guard checking `projects.user_id = auth.uid()` before running vector similarity queries, denying cross-student data leakage.
- 3. Preventing Quiz Grounding Contamination:** *Issue:* LLM quiz generation prompts without material scoping pulled concepts from unrelated project specification files. *Human Fix:* Enforced strict `source_material_id` scoping in queries and implemented an automated `FORBIDDEN_META_KEYWORDS` filter.
- 4. Resilient Supabase Client Connection:** *Issue:* Initial local signup attempts triggered browser `Failed to fetch` due to placeholder `.env.local` Supabase URLs. *Human Fix:* Guided configuration of real Supabase project credentials, validated the connection via Node.js fetch tests, and verified proper cookie persistence.

5. Development Velocity & Quality Metrics

Engineering Dimension	Traditional Estimate	AI-Assisted Time	Acceleration Factor	Verification Method
Relational Schema & Migrations	12 hours	3 hours	4.0×	Executed in clean Supabase project, 16 tables verified.
Full-Stack UI Scaffolding	24 hours	5 hours	4.8×	Interactive testing across dashboard, chat, quiz, growth.
RAG & Vector Pipeline	16 hours	4 hours	4.0×	Tested with Cryptography lecture notes, 0.70 threshold.
Adaptive Learning Logic	14 hours	3 hours	4.6×	7-signal algorithm verified across 16 test fixtures.
Test Suite Development	18 hours	3.5 hours	5.1×	128 automated unit/integration tests running in < 35s.
Total Engineering Effort	84 hours	18.5 hours	4.5×	Full project completion with zero technical debt.

6. Conclusion & Integrity Statement

AI tools served as an indispensable partner throughout the creation of the **AI Study Companion**, enabling rapid exploration of complex cognitive models and rigorous test coverage. Every line of generated code, prompt instruction, and architectural contract was audited, refined, and validated by the candidate to guarantee a production-grade, secure, and resilient learning platform.